

1 資料結構

1.1 DSU

```
struct DSU {
    vector<int> f, sz;
    int cnt;
    DSU(int n) : f(n), sz(n) {
        for (int i = 0; i < n; i++) {
            f[i] = i;
            sz[i] = 1;
        }
    }
    int find(int x) {
        if (x == f[x]) return x;
        return f[x] = find(f[x]);
    }
    void merge(int x, int y) {
        x = find(x); y = find(y);
        if (x == y) return;
        if (sz[x] < sz[y])
            swap(x, y);
        f[y] = x;
        sz[x] += sz[y];
        cnt--;
    }
    bool same(int a, int b) {
        return find(a) == find(b);
    }
    int groups() { // 找有幾群
        return cnt;
    }
};
```

1.2 BIT

```
struct Binary_Indexed_Tree{
    int n;
    vector<long long> bit;
    int lowbit(int x){
        return x&-x;
    }
    void init(int _n){
        n = _n+1;
        bit = vector<long long>(n,0);
    }
    void update(int x,int v){
        for(; x<n; x+=lowbit(x)){
            bit[x] += v;
        }
    }
    long long query(int x){
        long long ret = 0;
        for(; x>0; x-=lowbit(x)){
            ret += bit[x];
        }
        return ret;
    }
} BIT;
```

2 圖論

2.1 Dijkstra

```
#include <iostream>
#include <vector>
#include <utility>
#include <queue>
using namespace std;

#define pii pair<int,int>
#define INF 1e18
#define int long long

vector<pii> vec[100005];
```

```
vector<int> vis(100005);

int n, m;

void dijkstra(int s,int t){ // 起點 · 終點
    int dis[100005];
    for(int i=0;i<100005;i++){ // 初始化
        dis[i]=INF; // 值要設為比可能的最短路徑權重還要
        // 大的值
    }
    dis[s]=0;
    priority_queue<pii,vector<pii>,greater<pii>> pq;
    // 以小到大排序
    pq.push({dis[s],s});
    while(pq.empty()==0){
        int u=pq.top().second;
        pq.pop();
        if (vis[u]) continue;
        vis[u]=1;

        for (int i = 0; i < vec[u].size(); i++) {
            int v = vec[u][i].first;
            int w = vec[u][i].second;
            if(dis[u]+w < dis[v]){ // 鬆弛
                dis[v] = dis[u]+w;
                pq.push({dis[v],v});
            }
        }
    }

    for (int i = 1; i <= n; i++) {
        cout << dis[i] << " ";
    }
}

main() {
    cin >> n >> m;
    for (int i = 0; i < m; i++) {
        int a, b, c;
        cin >> a >> b >> c;
        vec[a].push_back({b, c});
    }

    dijkstra(1, -1);
}
```

2.2 SPTree

```
#include <iostream>
#include <vector>
#include <utility>
#include <queue>
#include <tuple>
using namespace std;

#define pii pair<int,int>
#define INF 1e18
#define int long long

vector<tuple<int, int, int>> vec[200005];
vector<int> vis(200005);
vector<int> path(200005);

int n, m;

void dijkstra(int s,int t){ // 起點 · 終點
    int dis[200005];
    for(int i=0;i<200005;i++){ // 初始化
        dis[i]=INF; // 值要設為比可能的最短路徑權重還要
        // 大的值
    }
    dis[s]=0;
    priority_queue<pii,vector<pii>,greater<pii>> pq;
    // 以小到大排序
    pq.push({dis[s],s});
    while(pq.empty()==0){
        int u=pq.top().second;
        pq.pop();
        if (vis[u]) continue;
```

```

        vis[u]=1;

        for (auto [v, w, i] : vec[u]) {
            if(dis[u]+w < dis[v]){//鬆弛
                dis[v] = dis[u]+w;
                pq.push({dis[v],v});
                path[v] = i;
            }
        }
    }

    for (int i = 2; i <= n; i++) {
        cout << path[i] << " ";
    }
}

main() {
    cin >> n >> m;
    for (int i = 1; i <= m; i++) {
        int a, b, c;
        cin >> a >> b >> c;
        vec[a].push_back({b, c, i});
        vec[b].push_back({a, c, i});
    }

    dijkstra(1, -1);
}

```

2.3 Bellman

```

#include <iostream>
#include <vector>
#include <utility>
#include <queue>
#include <tuple>
#include <climits>
using namespace std;

#define int long long

vector<tuple<int, int, int>> edge[2005];
int n, m;

void bellman(int s, int t) {
    int dis[2005];
    for (int i = 0; i < 2005; i++) dis[i] = LLONG_MIN;
    dis[s] = 0;

    for (int iter = 1; iter <= n; iter++) {
        vector<int> chang;
        for (int u = 1; u <= n; u++) {
            if (dis[u] == LLONG_MIN) continue;
            for (auto [from, v, w] : edge[u]) {
                if (dis[u] + w > dis[v]) {
                    dis[v] = dis[u] + w;
                    if (iter == n) chang.push_back(v);
                }
            }
        }
    }

    if (iter == n) {
        queue<int> q;
        vector<int> vis(2005);
        for (auto i : chang) q.push(i);

        while (!q.empty()) {
            int u = q.front(); q.pop();
            if (u == n) {
                cout << "inf";
                exit(0);
            }
            for (auto [u, v, w] : edge[u]) {
                if (!vis[v]) {
                    vis[v] = true;
                    q.push(v);
                }
            }
        }
    }
}

```

```

    }

    cout << dis[n];
}

signed main() {
    cin >> n >> m;
    for (int i = 1; i <= m; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        edge[u].push_back({u, v, w});
    }
    bellman(1, -1);
}

```

3 數學

3.1 Fast Power

```

int ksm(int x, int y) {
    int ans = 1;
    while (y) {
        if (y & 1)
            ans = ans * x;
        x = x * x;
        y >>= 1;
    }
    return ans;
}

// 矩陣乘法
vector<vector<int>> mul(const vector<vector<int>> &A,
    const vector<vector<int>> &B) {
    int n = A.size();
    vector<vector<int>> C(n, vector<int>(n, 0));

    for (int i = 0; i < n; i++) {
        for (int k = 0; k < n; k++) if (A[i][k]) {
            for (int j = 0; j < n; j++) {
                C[i][j] = (C[i][j] + A[i][k] * B[k][j]
                    ) % MOD;
            }
        }
    }
    return C;
}

// 矩陣快速幂
vector<vector<int>> ksm(vector<vector<int>> base, int
    exp) {
    int n = base.size();
    vector<vector<int>> res(n, vector<int>(n, 0));

    for (int i = 0; i < n; i++) res[i][i] = 1;

    while (exp > 0) {
        if (exp & 1) res = mul(res, base);
        base = mul(base, base);
        exp >>= 1;
    }
    return res;
}

```

4 字串

4.1 KMP

```

vector<int> build_lps(string s){
    int n = s.size();
    vector<int> lps(n);
    for(int i=1,j=0;i<n;i++){
        while(j>0 && s[i]!=s[j]) j=lps[j-1];
        if(s[i]==s[j]) j++;
        lps[i]=j;
    }
    return lps;
}

```